AI Assisted Coding

Assignment 12.3

Name: Vivek

Hallticket:2303A52009

Task_01 :

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives:

- Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
- Compare different algorithms in terms of efficiency and use cases.
- Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
  - Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
  - Ask AI to include time complexity and space complexity in the function docstring.
  - Verify the generated code with test cases.
- Expected Output:
  - A functional Python script implementing Merge Sort with proper documentation.

Prompt: Generate a Python program that implements the Merge Sort algorithm.
Requirements:
1. Create a function merge_sort(arr) that sorts a list in ascending order using recursion.
2. Include a helper function merge(left, right) to merge two sorted lists.
3. Add a proper Python docstring in merge_sort() explaining:
   - What the function does
   - Parameters
   - Return value
   - Time Complexity
   - Space Complexity
4. Write a small main section that tests the function using sample test cases.
5. Print the original list and the sorted list.

Code:

```python
def merge_sort(arr):
    """
    Sorts a list using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new list containing the sorted elements in ascending order.

    Time Complexity:
    - Best Case: O(n log n)
    - Average Case: O(n log n)
    - Worst Case: O(n log n)

    Space Complexity:
    - O(n) auxiliary space for merging.
    """
    # Base case: A list of size 0 or 1 is already sorted
    if len(arr) <= 1:
        return arr
    # Divide the list into two halves
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-As
Original List: [38, 27, 43, 3, 9, 82, 10]
Sorted List: [3, 9, 10, 27, 38, 43, 82]

Original List: [5, 2, 9, 1, 5, 6]
Sorted List: [1, 2, 5, 5, 6, 9]

Original List: [1, 2, 3, 4, 5]
Sorted List: [1, 2, 3, 4, 5]
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
  - Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
  - Include docstrings explaining best, average, and worst-case complexities.
  - Test with various inputs.
- Expected Output:
  - Python code implementing binary search with AI-generated comments and docstrings.

Prompt: Generate a Python program that implements the Binary Search algorithm.
Requirements:
1. Create a function binary_search(arr, target) that searches for a target value in a sorted list.
2. The function should return the index of the target element if found, otherwise return -1.

3. Add a proper Python docstring explaining:
   - Purpose of the function
   - Parameters
   - Return value
   - Best case, average case, and worst case time complexity
4. Include inline comments explaining key steps of the algorithm.
5. Add a main section with test cases to verify the function.
6. Print whether the target element was found and its index.

Code:

```python
def binary_search(arr, target):
    target (int/float): The value to search for.
    Returns:
    int: Index of the target element if found, otherwise -1.
    Time Complexity:
    Best Case: O(1)      -> Target found at the middle element immediately.
    Average Case: O(log n)
    Worst Case: O(log n) -> Search space halves every iteration.
    Space Complexity:
    O(1) for the iterative implementation.
    """
    # Initialize the search boundaries
    left = 0
    right = len(arr) - 1
    # Continue searching while the search range is valid
    while left <= right:
        # Find the middle index
        mid = (left + right) // 2
        # Check if the middle element is the target
        if arr[mid] == target:
            return mid
        # If target is greater, search the right half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, search the left half
```

Output:

```
Sorted List: [1, 2, 3, 4, 5]
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Co
Array: [2, 5, 7, 10, 14, 18, 21, 30]
Element 14 found at index 4
Element 9 not found
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Prompt: Create a Python program for a Smart Healthcare Appointment Scheduling System.
Requirements:
1. Each appointment record should contain:
   - Appointment ID
   - Patient Name
   - Doctor Name
   - Appointment Time
   - Consultation Fee

2. Implement efficient algorithms for:
   - Searching an appointment using Appointment ID
   - Sorting appointments by Appointment Time
   - Sorting appointments by Consultation Fee
3. Use the following algorithms:
   - Binary Search for searching by Appointment ID
   - Merge Sort for sorting by Appointment Time
   - Quick Sort for sorting by Consultation Fee

4. Include comments explaining the algorithm logic.
5. Print:
   - Result of searching for a specific appointment
   - Appointments sorted by time
   - Appointments sorted by consultation fee.

Code:

```python
class Appointment:
    def __init__(self, aid, patient, doctor, time, fee):
        self.aid = aid
        self.patient = patient
        self.doctor = doctor
        self.time = time
        self.fee = fee
    def __repr__(self):
        return f"{self.aid} | {self.patient} | {self.doctor} | Time:{self.time} | Fee:{self.fee}"
# Binary Search by Appointment ID
def binary_search(appointments, target_id):
    low = 0
    high = len(appointments) - 1
    while low <= high:
        mid = (low + high) // 2
        if appointments[mid].aid == target_id:
            return appointments[mid]
        elif appointments[mid].aid < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Merge Sort by Appointment Time
def merge_sort_time(arr):
    if len(arr) <= 1:
```

Output:

```
Element 9 not found
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Co
Search Appointment by ID:
103 | Charlie | Dr. Lee | Time:08:45 | Fee:400

Appointments Sorted by Time:
103 | Charlie | Dr. Lee | Time:08:45 | Fee:400
101 | Alice | Dr. Smith | Time:09:00 | Fee:500
105 | Eva | Dr. Brown | Time:09:30 | Fee:450
102 | Bob | Dr. John | Time:10:30 | Fee:700
104 | David | Dr. Smith | Time:11:15 | Fee:600

Appointments Sorted by Fee:
103 | Charlie | Dr. Lee | Time:08:45 | Fee:400
105 | Eva | Dr. Brown | Time:09:30 | Fee:450
101 | Alice | Dr. Smith | Time:09:00 | Fee:500
104 | David | Dr. Smith | Time:11:15 | Fee:600
102 | Bob | Dr. John | Time:10:30 | Fee:700
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python

Prompt: Create a Python program for a Railway Ticket Reservation System.

Requirements:

1. Each booking record should contain:
   - Ticket ID
   - Passenger Name
   - Train Number
   - Seat Number
   - Travel Date
2. Implement efficient algorithms for:
   - Searching tickets by Ticket ID
   - Sorting bookings by Travel Date
   - Sorting bookings by Seat Number
3. Use the following algorithms:
   - Binary Search for searching by Ticket ID
   - Merge Sort for sorting by Travel Date
   - Quick Sort for sorting by Seat Number
4. Include comments explaining the algorithm logic.
5. Print:
   - Result of searching a ticket
   - Bookings sorted by travel date
   - Bookings sorted by seat number.

Code:

```python
# Ticket class
class Ticket:
    def __init__(self, tid, passenger, train_no, seat_no, date):
        self.tid = tid
        self.passenger = passenger
        self.train_no = train_no
        self.seat_no = seat_no
        self.date = date
    def __repr__(self):
        return f"{self.tid} | {self.passenger} | Train:{self.train_no} | Seat:{self.seat_no} | Date:{self.date}"
# Binary Search by Ticket ID
def binary_search(tickets, target_id):
    low = 0
    high = len(tickets) - 1
    while low <= high:
        mid = (low + high) // 2

        if tickets[mid].tid == target_id:
            return tickets[mid]
        elif tickets[mid].tid < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Merge Sort by Travel Date
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-A
Search Ticket by ID:
203 | Kiran | Train:12005 | Seat:30 | Date:2026-03-18

Tickets Sorted by Travel Date:
204 | Meena | Train:12627 | Seat:5 | Date:2026-03-11
202 | Anita | Train:12628 | Seat:12 | Date:2026-03-12
205 | Arjun | Train:12005 | Seat:21 | Date:2026-03-14
201 | Rahul | Train:12627 | Seat:45 | Date:2026-03-15
203 | Kiran | Train:12005 | Seat:30 | Date:2026-03-18

Tickets Sorted by Seat Number:
204 | Meena | Train:12627 | Seat:5 | Date:2026-03-11
202 | Anita | Train:12628 | Seat:12 | Date:2026-03-12
205 | Arjun | Train:12005 | Seat:21 | Date:2026-03-14
203 | Kiran | Train:12005 | Seat:30 | Date:2026-03-18
201 | Rahul | Train:12627 | Seat:45 | Date:2026-03-15
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

Task Description #5: Smart Hostel Room Allocation System
A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.

Prompt: You are a computer science expert.
Design a Python program for a **Smart Hostel Room Allocation System**.
The system stores records containing:

- Student ID
- Room Number
- Floor
- Allocation Date

The program must perform the following operations:

1. **Search allocation details using Student ID** efficiently.
2. **Sort records based on Room Number or Allocation Date**.

Requirements:

- Suggest optimized algorithms for searching and sorting.
- Use **Binary Search** for searching (after sorting by Student ID).
- Use **Python's built-in sorting (Timsort)** for sorting records.
- Provide justification for the algorithm choices.
- Provide a clean and complete Python implementation with sample data.
- Display results clearly.

Code:

```python
from datetime import datetime
# Hostel allocation records
records = [
    {"student_id": 101, "room": 205, "floor": 2, "date": "2024-07-01"},
    {"student_id": 103, "room": 101, "floor": 1, "date": "2024-06-20"},
    {"student_id": 102, "room": 303, "floor": 3, "date": "2024-07-10"},
    {"student_id": 104, "room": 202, "floor": 2, "date": "2024-06-15"}
]
# Convert date string to datetime
for r in records:
    r["date"] = datetime.strptime(r["date"], "%Y-%m-%d")
# -------- Binary Search --------
def binary_search(data, target):
    low = 0
    high = len(data) - 1
    while low <= high:
        mid = (low + high) // 2
        if data[mid]["student_id"] == target:
            return data[mid]
        elif data[mid]["student_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None
```

Output:

```
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\Code Files'; &
\.venv\Scripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-
962' '--' 'D:\3-2\AI Assitant Coding\Code Files\Task-12.5.py'
Original Records
{'student_id': 102, 'room': 303, 'floor': 3, 'date': datetime.datetime(2024, 7, 10, 0, 0)}
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}

Enter Student ID to search: 104

Student Allocation Found:
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}

Sorted by Room Number:
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}
{'student_id': 101, 'room': 205, 'floor': 2, 'date': datetime.datetime(2024, 7, 1, 0, 0)}
{'student_id': 102, 'room': 303, 'floor': 3, 'date': datetime.datetime(2024, 7, 10, 0, 0)}

Sorted by Allocation Date:
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
{'student_id': 101, 'room': 205, 'floor': 2, 'date': datetime.datetime(2024, 7, 1, 0, 0)}
{'student_id': 102, 'room': 303, 'floor': 3, 'date': datetime.datetime(2024, 7, 10, 0, 0)}
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.

Implement Python functions.

Prompt: You are a computer science expert.

Create a Python program for an **Online Movie Streaming Platform** that stores movie records containing:

- Movie ID

- Title
- Genre
- Rating
- Release Year

The system must perform the following operations:

1. Search for a movie using **Movie ID** efficiently.
2. Sort movies based on **Rating** or **Release Year**.

Requirements:

- Recommend appropriate searching and sorting algorithms.
- Use **Binary Search** for searching after sorting by Movie ID.
- Use Python's built-in **Timsort (sorted or sort)** for sorting.
- Provide justification for the chosen algorithms.
- Implement clean Python functions for searching and sorting.
- Include sample movie records and display results clearly.

Code:

```python
# Movie records
movies = [
    {"movie_id": 101, "title": "Inception", "genre": "Sci-Fi", "rating": 8.8, "year": 2010},
    {"movie_id": 105, "title": "Avengers Endgame", "genre": "Action", "rating": 8.4, "year": 2019},
    {"movie_id": 103, "title": "Interstellar", "genre": "Sci-Fi", "rating": 8.6, "year": 2014},
    {"movie_id": 102, "title": "Joker", "genre": "Drama", "rating": 8.5, "year": 2019},
    {"movie_id": 104, "title": "The Dark Knight", "genre": "Action", "rating": 9.0, "year": 2008}
]
# Binary Search for Movie ID
def search_movie(movies, target):
    low = 0
    high = len(movies) - 1
    while low <= high:
        mid = (low + high) // 2
        if movies[mid]["movie_id"] == target:
            return movies[mid]
        elif movies[mid]["movie_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Sort movies by rating
def sort_by_rating(movies):
    return sorted(movies, key=lambda x: x["rating"], reverse=True)
```

Output:

```
{'movie_id': 102, 'title': 'Joker', 'genre': 'Drama', 'rating': 8.5, 'year': 2019}
{'movie_id': 104, 'title': 'The Dark Knight', 'genre': 'Action', 'rating': 9.0, 'year': 2008}

Enter Movie ID to search: 101

Movie Found:
{'movie_id': 101, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}

Movies Sorted by Rating:
{'movie_id': 104, 'title': 'The Dark Knight', 'genre': 'Action', 'rating': 9.0, 'year': 2008}
{'movie_id': 101, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}
{'movie_id': 103, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}
{'movie_id': 102, 'title': 'Joker', 'genre': 'Drama', 'rating': 8.5, 'year': 2019}
{'movie_id': 105, 'title': 'Avengers Endgame', 'genre': 'Action', 'rating': 8.4, 'year': 2019}

Movies Sorted by Release Year:
{'movie_id': 104, 'title': 'The Dark Knight', 'genre': 'Action', 'rating': 9.0, 'year': 2008}
{'movie_id': 101, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}
{'movie_id': 103, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}
{'movie_id': 102, 'title': 'Joker', 'genre': 'Drama', 'rating': 8.5, 'year': 2019}
{'movie_id': 105, 'title': 'Avengers Endgame', 'genre': 'Action', 'rating': 8.4, 'year': 2019}
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Prompt: You are a computer science expert.
Design a Python program for a **Smart Agriculture Crop Monitoring System**.
The system stores crop data with the following fields:

- Crop ID
- Crop Name
- Soil Moisture Level
- Temperature
- Yield Estimate

The system must perform the following operations:

1. Search crop details using **Crop ID** efficiently.
2. Sort crops based on **Soil Moisture Level** or **Yield Estimate**.

Requirements:

- Recommend appropriate searching and sorting algorithms.
- Use **Binary Search** for searching after sorting by Crop ID.
- Use Python's built-in **Timsort (sorted / sort)** for sorting.
- Provide justification for the algorithm choices.
- Implement clear Python functions for searching and sorting.
- Include sample crop records and display the results clearly.

Code:

```python
# Crop monitoring records
crops = [
    {"crop_id": 101, "name": "Wheat", "moisture": 45, "temperature": 28, "yield": 3.5},
    {"crop_id": 105, "name": "Rice", "moisture": 70, "temperature": 30, "yield": 4.2},
    {"crop_id": 103, "name": "Corn", "moisture": 55, "temperature": 27, "yield": 3.8},
    {"crop_id": 102, "name": "Barley", "moisture": 40, "temperature": 26, "yield": 2.9},
    {"crop_id": 104, "name": "Soybean", "moisture": 60, "temperature": 29, "yield": 3.6}
]
# -------- Binary Search --------
def search_crop(crops, target):
    low = 0
    high = len(crops) - 1
    while low <= high:
        mid = (low + high) // 2
        if crops[mid]["crop_id"] == target:
            return crops[mid]
        elif crops[mid]["crop_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None
# -------- Sorting Functions --------
def sort_by_moisture(crops):
    return sorted(crops, key=lambda x: x["moisture"])
def sort_by_yield(crops):
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Task-12.5.py"
Original Records
{'student_id': 101, 'room': 205, 'floor': 2, 'date': datetime.datetime(2024, 7, 1, 0, 0)}
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
{'student_id': 102, 'room': 303, 'floor': 3, 'date': datetime.datetime(2024, 7, 10, 0, 0)}
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}

Records Sorted by Student ID (for Binary Search)
{'student_id': 101, 'room': 205, 'floor': 2, 'date': datetime.datetime(2024, 7, 1, 0, 0)}
{'student_id': 102, 'room': 303, 'floor': 3, 'date': datetime.datetime(2024, 7, 10, 0, 0)}
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}

Enter Student ID to search: 101

Student Allocation Found:
{'student_id': 101, 'room': 205, 'floor': 2, 'date': datetime.datetime(2024, 7, 1, 0, 0)}

Sorted by Room Number:
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}
{'student_id': 101, 'room': 205, 'floor': 2, 'date': datetime.datetime(2024, 7, 1, 0, 0)}
{'student_id': 102, 'room': 303, 'floor': 3, 'date': datetime.datetime(2024, 7, 10, 0, 0)}

Sorted by Allocation Date:
{'student_id': 104, 'room': 202, 'floor': 2, 'date': datetime.datetime(2024, 6, 15, 0, 0)}
{'student_id': 103, 'room': 101, 'floor': 1, 'date': datetime.datetime(2024, 6, 20, 0, 0)}
Ready                                   Ln 375, Col 32   Spaces: 4   UTF-8   CRLF   { } Python
```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Prompt: You are a computer science expert

Create a Python program for an **Airport Flight Management System**.
The system stores flight information with the following fields:

- Flight ID
- Airline Name
- Departure Time
- Arrival Time
- Status

The system must perform these operations:

1. Search flight details using **Flight ID**.
2. Sort flights based on **Departure Time** or **Arrival Time**.

Requirements:

- Recommend suitable searching and sorting algorithms.
- Use **Binary Search** for searching after sorting flights by Flight ID.
- Use Python's built-in **Timsort (sorted or sort)** for sorting.
- Provide a brief justification for the chosen algorithms.
- Implement Python functions for searching and sorting.
- Include sample flight records and print results clearly.

Code:

```python
flights = [
    {"flight_id": 101, "airline": "IndiGo", "departure": "10:30", "arrival": "12:45", "status": "On Time"},
    {"flight_id": 105, "airline": "Air India", "departure": "09:15", "arrival": "11:30", "status": "Delayed"},
    {"flight_id": 103, "airline": "SpiceJet", "departure": "14:00", "arrival": "16:10", "status": "On Time"},
    {"flight_id": 102, "airline": "Vistara", "departure": "08:45", "arrival": "10:50", "status": "Boarding"},
    {"flight_id": 104, "airline": "Akasa Air", "departure": "12:20", "arrival": "14:30", "status": "On Time"}
]
# -------- Binary Search --------
def search_flight(flights, target):
    low = 0
    high = len(flights) - 1
    while low <= high:
        mid = (low + high) // 2
        if flights[mid]["flight_id"] == target:
            return flights[mid]
        elif flights[mid]["flight_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None
# -------- Sorting Functions --------
def sort_by_departure(flights):
    return sorted(flights, key=lambda x: x["departure"])
def sort_by_arrival(flights):
    return sorted(flights, key=lambda x: x["arrival"])
```

Output:

```
{'flight_id': 102, 'airline': 'Vistara', 'departure': '08:45', 'arrival': '10:50', 'status': 'Boarding'}
{'flight_id': 104, 'airline': 'Akasa Air', 'departure': '12:20', 'arrival': '14:30', 'status': 'On Time'}

Enter Flight ID to search: 102

Flight Found:
{'flight_id': 102, 'airline': 'Vistara', 'departure': '08:45', 'arrival': '10:50', 'status': 'Boarding'}

Flights Sorted by Departure Time:
{'flight_id': 102, 'airline': 'Vistara', 'departure': '08:45', 'arrival': '10:50', 'status': 'Boarding'}
{'flight_id': 105, 'airline': 'Air India', 'departure': '09:15', 'arrival': '11:30', 'status': 'Delayed'}
{'flight_id': 101, 'airline': 'IndiGo', 'departure': '10:30', 'arrival': '12:45', 'status': 'On Time'}
{'flight_id': 104, 'airline': 'Akasa Air', 'departure': '12:20', 'arrival': '14:30', 'status': 'On Time'}
{'flight_id': 103, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:10', 'status': 'On Time'}

Flights Sorted by Arrival Time:
{'flight_id': 102, 'airline': 'Vistara', 'departure': '08:45', 'arrival': '10:50', 'status': 'Boarding'}
{'flight_id': 105, 'airline': 'Air India', 'departure': '09:15', 'arrival': '11:30', 'status': 'Delayed'}
{'flight_id': 101, 'airline': 'IndiGo', 'departure': '10:30', 'arrival': '12:45', 'status': 'On Time'}
{'flight_id': 104, 'airline': 'Akasa Air', 'departure': '12:20', 'arrival': '14:30', 'status': 'On Time'}
{'flight_id': 103, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:10', 'stat
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

d:\3-2\AI Assitant Coding\Code File